cs418-hw1-F24

September 19, 2024

1 Homework 1

UIC CS 418, Fall 2024

*According to the **Academic Integrity Policy** of this course, all work submitted for grading must be done individually, unless otherwise specified. While we encourage you to talk to your peers and learn from them, this interaction must be superficial with regards to all work submitted for grading. This means you cannot work in teams, you cannot work side-by-side, you cannot submit someone else's work (partial or complete) as your own. In particular, note that you are guilty of academic dishonesty if you extend or receive any kind of unauthorized assistance. Absolutely no transfer of program code between students is permitted (paper or electronic), and you may not solicit code from family, friends, or online forums. Other examples of academic dishonesty include emailing your program to another student, copying-pasting code from the internet, working in a group on a homework assignment, and allowing a tutor, TA, or another individual to write an answer for you. Academic dishonesty is unacceptable, and penalties range from failure to expulsion from the university; cases are handled via the official student conduct process described at https://dos.uic.edu/conductforstudents.shtml.*

1.1 Due Date

This assignment is due at 11:59pm CST on September 19, 2024. All parts of the assignments are due at the same time. If any segment of the assignment is submitted late, the late submission policy applies for the whole assignment. Instructions on how to submit it to Gradescope are given at the end of the notebook and should be followed carefully.

1.2 Part 1 (45% of HW1): Data processing with pandas

In this homework you will see examples of some commonly used data wrangling tools in Python. In particular, we aim to give you some familiarity with:

- Slicing data frames
- Filtering data
- Grouped counts
- Joining two tables
- NA/Null values

1.3 Part 1: Practice (15%)

This part of the homework is graded manually based on showing the correct outputs after executing each step.

1.4 Setup

You need to execute each step (run each Cell), in order for the next ones to work. First, import necessary libraries:

[383]:
```
import pandas as pd
import numpy as np
```

The code below produces the data frames used in the examples:

[384]:
```
heroes = pd.DataFrame(
    data={'color': ['red', 'green', 'black',
                    'blue', 'black', 'red'],
          'first_seen_on': ['a', 'a', 'f', 'a', 'a', 'f'],
          'first_season': [2, 1, 2, 3, 3, 1]},
    index=['flash', 'arrow', 'vibe',
           'atom', 'canary', 'firestorm']
)

identities = pd.DataFrame(
    data={'ego': ['barry allen', 'oliver queen', 'cisco ramon',
                  'ray palmer', 'sara lance',
                  'martin stein', 'ronnie raymond'],
          'alter-ego': ['flash', 'arrow', 'vibe', 'atom',
                        'canary', 'firestorm', 'firestorm']}
)

teams = pd.DataFrame(
    data={'team': ['flash', 'arrow', 'flash', 'legends',
                   'flash', 'legends', 'arrow'],
          'hero': ['flash', 'arrow', 'vibe', 'atom',
                   'killer frost', 'firestorm', 'speedy']})
```

1.5 Pandas and Wrangling

For the examples that follow, we will be using a toy data set containing information about superheroes in the Arrowverse. In the `first_seen_on` column, `a` stands for Archer and `f`, Flash.

[385]:
```
heroes
```

[385]:

	color	first_seen_on	first_season
flash	red	a	2
arrow	green	a	1
vibe	black	f	2
atom	blue	a	3
canary	black	a	3
firestorm	red	f	1

[386]:
```
identities
```

[386]:
```
              ego  alter-ego
0     barry allen      flash
1    oliver queen      arrow
2     cisco ramon       vibe
3      ray palmer       atom
4      sara lance     canary
5    martin stein  firestorm
6  ronnie raymond  firestorm
```

[387]:
```
teams
```

[387]:
```
      team          hero
0    flash         flash
1    arrow         arrow
2    flash          vibe
3  legends          atom
4    flash  killer frost
5  legends     firestorm
6    arrow        speedy
```

1.5.1 Slice and Dice

Column selection by label To select a column of a `DataFrame` by column label, the safest and fastest way is to use the `.loc` method. General usage looks like `frame.loc[rowname,colname]`. (Reminder that the colon `:` means "everything"). For example, if we want the `color` column of the `heroes` data frame, we would use :

[388]:
```
heroes.loc[:, 'color']
```

[388]:
```
flash          red
arrow        green
vibe         black
atom          blue
canary       black
firestorm      red
Name: color, dtype: object
```

Selecting multiple columns is easy. You just need to supply a list of column names. Here we select the color and value columns:

[389]:
```
heroes.loc[:, ['color', 'first_season']]
```

[389]:
```
           color  first_season
flash        red             2
arrow      green             1
vibe       black             2
atom        blue             3
canary     black             3
```

```
firestorm    red             1
```

While .loc is invaluable when writing production code, it may be a little too verbose for interactive use. One recommended alternative is the [] method, which takes on the form frame[‘colname’].

[390]:
```
heroes['first_seen_on']
```

[390]:
```
flash        a
arrow        a
vibe         f
atom         a
canary       a
firestorm    f
Name: first_seen_on, dtype: object
```

Row Selection by Label Similarly, if we want to select a row by its label, we can use the same .loc method.

[391]:
```
heroes.loc[['flash', 'vibe'], :]
```

[391]:
```
       color first_seen_on  first_season
flash    red             a             2
vibe   black             f             2
```

If we want all the columns returned, we can, for brevity, drop the colon without issue.

[392]:
```
heroes.loc[['flash', 'vibe']]
```

[392]:
```
       color first_seen_on  first_season
flash    red             a             2
vibe   black             f             2
```

General Selection by Label More generally you can slice across both rows and columns at the same time. For example:

[393]:
```
heroes.loc['flash':'atom', :'first_seen_on']
```

[393]:
```
       color first_seen_on
flash    red             a
arrow  green             a
vibe   black             f
atom    blue             a
```

Selection by Integer Index If you want to select rows and columns by position, the Data Frame has an analogous `.iloc` method for integer indexing. Remember that Python indexing starts at 0.

[394]:
```
heroes.iloc[:4,:2]
```

[394]:
```
        color first_seen_on
flash     red            a
arrow   green            a
vibe    black            f
atom     blue            a
```

1.5.2 Filtering with boolean arrays

Filtering is the process of removing unwanted material. In your quest for cleaner data, you will undoubtedly filter your data at some point: whether it be for clearing up cases with missing values, culling out fishy outliers, or analyzing subgroups of your data set. For example, we may be interested in characters that debuted in season 3 of Archer. Note that compound expressions have to be grouped with parentheses.

[395]:
```
heroes[(heroes['first_season']==3) & (heroes['first_seen_on']=='a')]
```

[395]:
```
         color first_seen_on  first_season
atom      blue            a             3
canary  black             a             3
```

Problem Solving Strategy We want to highlight the strategy for filtering to answer the question above:

- **Identify the variables of interest**
  - Interested in the debut: `first_season` and `first_seen_on`
- **Translate the question into statements one with True/False answers**
  - Did the hero debut on Archer? → The hero has `first_seen_on` equal to `a`
  - Did the hero debut in season 3? → The hero has `first_season` equal to `3`
- **Translate the statements into boolean statements**
  - The hero has `first_seen_on` equal to `a` → `hero['first_seen_on']=='a'`
  - The hero has `first_season` equal to `3` → `heroes['first_season']==3`
- **Use the boolean array to filter the data**

Note that compound expressions have to be grouped with parentheses.

For your reference, some commonly used comparison operators are given below.

Symbol	Usage	Meaning
==	a == b	Does a equal b?
<=	a <= b	Is a less than or equal to b?
>=	a >= b	Is a greater than or equal to b?
<	a < b	Is a less than b?
>	a > b	Is a greater than b?
~	~p	Returns negation of p
|	p | q	p OR q
&	p & q	p AND q
^	p ^ q	p XOR q (exclusive or)

An often-used operation missing from the above table is a test-of-membership. The `Series.isin(values)` method returns a boolean array denoting whether each element of `Series` is in `values`. We can then use the array to subset our data frame. For example, if we wanted to see which rows of `heroes` had values in $\{1, 3\}$, we would use:

[396]:
```
heroes[heroes['first_season'].isin([1,3])]
```

[396]:
```
           color first_seen_on  first_season
arrow      green             a             1
atom        blue             a             3
canary     black             a             3
firestorm    red             f             1
```

Notice that in both examples above, the expression in the brackets evaluates to a boolean series. The general strategy for filtering data frames, then, is to write an expression of the form `frame[logical statement]`.

1.5.3 Counting Rows

To count the number of instances of a value in a `Series`, we can use the `value_counts` method. Below we count the number of instances of each color.

[397]:
```
heroes['color'].value_counts()
```

[397]:
```
color
red      2
black    2
green    1
blue     1
Name: count, dtype: int64
```

A more sophisticated analysis might involve counting the number of instances a tuple appears. Here we count $(color, value)$ tuples.

[398]:
```
heroes.groupby(['color', 'first_season']).size()
```

[398]:
```
color  first_season
black  2               1
       3               1
blue   3               1
green  1               1
red    1               1
       2               1
dtype: int64
```

This returns a series that has been multi-indexed. We'll eschew this topic for now. To get a data frame back, we'll use the `reset_index` method, which also allows us to simulataneously name the new column.

[399]:
```
heroes.groupby(['color', 'first_season']).size().reset_index(name='count')
```

[399]:
	color	first_season	count
0	black	2	1
1	black	3	1
2	blue	3	1
3	green	1	1
4	red	1	1
5	red	2	1

1.5.4 Joining Tables on One Column

Suppose we have another table that classifies superheroes into their respective teams. Note that `canary` is not in this data set and that `killer frost` and `speedy` are additions that aren't in the original `heroes` set.

For simplicity of the example, we'll convert the index of the `heroes` data frame into an explicit column called `hero`. A careful examination of the documentation will reveal that joining on a mixture of the index and columns is possible.

[400]:
```
heroes['hero'] = heroes.index
heroes
```

[400]:
	color	first_seen_on	first_season	hero
flash	red	a	2	flash
arrow	green	a	1	arrow
vibe	black	f	2	vibe
atom	blue	a	3	atom
canary	black	a	3	canary
firestorm	red	f	1	firestorm

Inner Join The inner join below returns rows representing the heroes that appear in both data frames.

[401]:
```
pd.merge(heroes, teams, how='inner', on='hero')
```

[401]:
	color	first_seen_on	first_season	hero	team
0	red	a	2	flash	flash
1	green	a	1	arrow	arrow
2	black	f	2	vibe	flash
3	blue	a	3	atom	legends
4	red	f	1	firestorm	legends

Left and right join The left join returns rows representing heroes in the `heroes` ("left") data frame, augmented by information found in the `teams` data frame. Its counterpart, the right join, would return heroes in the `teams` data frame. Note that the `team` for hero `canary` is an `NaN` value, representing missing data.

[402]:
```
pd.merge(heroes, teams, how='left', on='hero')
```

[402]:

	color	first_seen_on	first_season	hero	team
0	red	a	2	flash	flash
1	green	a	1	arrow	arrow
2	black	f	2	vibe	flash
3	blue	a	3	atom	legends
4	black	a	3	canary	NaN
5	red	f	1	firestorm	legends

Outer join An outer join on `hero` will return all heroes found in both the left and right data frames. Any missing values are filled in with `NaN`.

[403]:
```
pd.merge(heroes, teams, how='outer', on='hero')
```

[403]:

	color	first_seen_on	first_season	hero	team
0	green	a	1.0	arrow	arrow
1	blue	a	3.0	atom	legends
2	black	a	3.0	canary	NaN
3	red	f	1.0	firestorm	legends
4	red	a	2.0	flash	flash
5	NaN	NaN	NaN	killer frost	flash
6	NaN	NaN	NaN	speedy	arrow
7	black	f	2.0	vibe	flash

More than one match? If the values in the columns to be matched don't uniquely identify a row, then a cartesian product is formed in the merge. For example, notice that `firestorm` has two different egos, so information from `heroes` had to be duplicated in the merge, once for each ego.

[404]:
```
pd.merge(heroes, identities, how='inner',
         left_on='hero', right_on='alter-ego')
```

[404]:

	color	first_seen_on	first_season	hero	ego	alter-ego
0	red	a	2	flash	barry allen	flash
1	green	a	1	arrow	oliver queen	arrow
2	black	f	2	vibe	cisco ramon	vibe
3	blue	a	3	atom	ray palmer	atom
4	black	a	3	canary	sara lance	canary
5	red	f	1	firestorm	martin stein	firestorm
6	red	f	1	firestorm	ronnie raymond	firestorm

1.5.5 Missing Values

There are a multitude of reasons why a data set might have missing values. The current implementation of Pandas uses the numpy NaN to represent these null values (older implementations even used `-inf` and `inf`). Future versions of Pandas might implement a true `null` value—keep your eyes peeled for this in updates! More information can be found http://pandas.pydata.org/pandas-docs/stable/user_guide/missing_data.html

Because of the specialness of missing values, they merit their own set of tools. Here, we will focus on detection. For replacement, see the docs.

[405]:
```
x = np.nan
y = pd.merge(heroes, teams, how='outer', on='hero')['first_season']
y
```

[405]:
```
0    1.0
1    3.0
2    3.0
3    1.0
4    2.0
5    NaN
6    NaN
7    2.0
Name: first_season, dtype: float64
```

To check if a value is null, we use the `isnull()` method for series and data frames. Alternatively, there is a `pd.isnull()` function as well.

[406]:
```
#x.isnull() # won't work since x is neither a series nor a data frame
```

[407]:
```
pd.isnull(x)
```

[407]:
```
True
```

[408]:
```
y.isnull()
```

[408]:
```
0    False
1    False
2    False
3    False
4    False
5     True
6     True
7    False
Name: first_season, dtype: bool
```

[409]:
```
pd.isnull(y)
```

[409]:
```
0    False
1    False
2    False
3    False
4    False
5     True
6     True
7    False
Name: first_season, dtype: bool
```

Since filtering out missing data is such a common operation, Pandas also has conveniently included the analogous `notnull()` methods and function for improved human readability.

[410]:
```
y.notnull()
```

[410]:
```
0     True
1     True
2     True
3     True
4     True
5    False
6    False
7     True
Name: first_season, dtype: bool
```

[411]:
```
y[y.notnull()]
```

[411]:
```
0    1.0
1    3.0
2    3.0
3    1.0
4    2.0
7    2.0
Name: first_season, dtype: float64
```

1.6 Part 1: Questions (30%)

The problems below is based on an article that appeared in https://chi.streetsblog.org/2022/04/25/data-analysis-found-cta-has-only-been-running-about-half-its-schedule-blue-line-runs

In short, during 2022, the train service run by CTA were somewhat irregular, especially since they were doing maintenance work. As mentioned in the article, Fabio Göttlicher, a resident of Chicago decided to record arrival times of the blue line at a particular train stop. We will use more recent data to calculate delays at this train stop. This data is given to you as 7 json files. You can see how you can do this by quering the CTA for real time feed in Fabio Göttlicher's website at https://github.com/FabioCZ/dude-wheres-my-train/tree/main

The first three parts of starter code below provides the skeleton for processing one json file with which you can calculate the delays.

In the last part, you will have to repeat the calculation for the full week and calculate weekly variance.

[412]:
```
# load the json file to get arrival and schedule for the day
arrival_data = pd.read_json('20240701.json')

# show the first few rows, by default 5
#print(arrival_data.head() )
print(arrival_data.shape)
```

```python
#let's look at the columns
print(arrival_data.columns)

#get arrivals and scheduled columns -- all actual arrival and scheduled times␣
↪are stored in these two dataframe columns
arrivals_info = arrival_data.loc[:,'arrivals']
schedule_info = arrival_data.loc[:,'scheduled']

print(arrivals_info.head())
print(schedule_info.head())
```

```
(27, 6)
Index(['date', 'arrivals', 'arrivalCt', 'scheduled', 'onTimePerformance',
       'uptimeLog'],
      dtype='object')
30111    [{'createdAt': '2024-06-30T23:50:41-05:00', 's…
30112    [{'createdAt': '2024-06-30T23:52:41-05:00', 's…
total                                                  NaN
0                                                      NaN
1                                                      NaN
Name: arrivals, dtype: object
30111    {'totalPerDay': 158, 'allDepartures': ['00:52:…
30112    {'totalPerDay': 156, 'allDepartures': ['00:27:…
total                                                  NaN
0                                                      NaN
1                                                      NaN
Name: scheduled, dtype: object
```

```
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/678236852.py:2:
FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings is
deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json('20240701.json')
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/678236852.py:2:
FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings is
deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json('20240701.json')
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/678236852.py:2:
FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings is
deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json('20240701.json')
```

1.6.1 Question 1.1 (8% credit)

Data Preprocessing to get scheduled and arrival times from JSON file

[413]:
```
# 4%credit
#extract schedule as a list

#allDepartures column has all the scheduled information
num_of_scheduled = len(schedule_info.iloc[1]["allDepartures"]) #change 1 to 0␣
 ↪to get schedule of 30111 or a particular direction (northbound)
print(num_of_scheduled)
## print the schedule of 30111
schedule_info_30111 = schedule_info.iloc[0]["allDepartures"]
print(schedule_info_30111)

#to get a particular scheduled time, we can use iloc
print(schedule_info.iloc[1]["allDepartures"][num_of_scheduled-5]) #prints the␣
 ↪5th from last entry
#observe that schedules are already stored in HH:MM:SS format, so no␣
 ↪preprocessing required

#insert to code create a list containing all the times when trains are␣
 ↪scheduled on the day
scheduled_times_list = []
scheduled_alltimes = schedule_info.iloc[1]["allDepartures"]
for scheduled_alltime in scheduled_alltimes:
    scheduled_times_list.append(scheduled_alltime)
scheduled_times = pd.DataFrame(scheduled_times_list)
print(num_of_scheduled==scheduled_times.shape[0]) #should evaluate to True
```

```
156
['00:52:30', '01:07:30', '01:22:30', '01:37:30', '01:52:30', '02:07:30',
'02:37:30', '03:07:30', '03:37:30', '04:07:30', '04:22:30', '04:37:30',
'04:52:30', '05:07:30', '05:22:30', '05:37:30', '05:52:30', '06:22:30',
'06:32:30', '06:42:30', '06:52:30', '07:02:30', '07:12:30', '07:15:30',
'07:22:30', '07:32:30', '07:42:30', '07:50:30', '07:58:00', '08:01:00',
'08:05:30', '08:13:00', '08:20:30', '08:24:00', '08:28:00', '08:35:30',
'08:43:00', '08:46:00', '08:50:30', '08:58:00', '09:05:30', '09:09:00',
'09:13:00', '09:28:00', '09:35:30', '09:43:00', '09:50:30', '09:58:00',
'10:13:00', '10:20:30', '10:28:00', '10:35:30', '10:43:00', '10:50:30',
'10:58:00', '11:05:30', '11:13:00', '11:20:30', '11:35:30', '11:43:00',
'11:50:30', '11:58:00', '12:05:30', '12:13:00', '12:20:30', '12:28:00',
'12:35:30', '12:43:00', '12:50:30', '12:58:00', '13:05:30', '13:13:00',
'13:20:30', '13:28:00', '13:35:30', '13:43:00', '13:50:30', '14:05:30',
'14:13:00', '14:20:30', '14:28:00', '14:35:30', '14:43:00', '14:50:30',
'14:58:00', '15:05:30', '15:13:00', '15:20:30', '15:35:30', '15:43:00',
'15:50:30', '15:56:30', '16:02:30', '16:08:30', '16:13:30', '16:18:30',
'16:23:30', '16:28:30', '16:33:30', '16:38:30', '16:43:30', '16:48:30',
```

```
'16:53:30', '16:58:30', '17:03:30', '17:08:30', '17:13:30', '17:18:30',
'17:23:30', '17:26:00', '17:28:30', '17:33:30', '17:38:30', '17:43:30',
'17:48:30', '17:53:30', '18:08:30', '18:14:30', '18:20:30', '18:28:00',
'18:35:30', '18:43:00', '18:58:00', '19:05:30', '19:13:00', '19:20:30',
'19:28:00', '19:35:30', '19:43:00', '19:50:30', '19:58:00', '20:05:30',
'20:13:30', '20:22:30', '20:32:30', '20:42:30', '20:52:30', '21:02:30',
'21:12:30', '21:22:30', '21:32:30', '21:42:30', '21:52:30', '22:02:30',
'22:12:30', '22:22:30', '22:42:30', '22:52:30', '23:02:30', '23:12:30',
'23:22:30', '23:32:30', '23:42:30', '23:52:30', '24:02:30', '24:12:30',
'24:22:30', '24:37:30']
23:12:00
True
```

[414]:

```
# 4%credit
#extract actual arrivals as a list

#it looks like all the arrivals are stored in the first and second rows
num_of_arrivals= len(arrivals_info.iloc[1]) #change 1 to 0 to get arrivals of␣
 ↪30111 or a particular direction (northbound)
print(num_of_arrivals) # gives number of arrival time in the direction
print(arrivals_info.iloc[1][num_of_arrivals-1]["arrival"])#extract last entry␣
 ↪in the series

## insert code to extract time in the string, that is, have to extract␣
 ↪substring between 'T' and '-'
def get_arrive_time(arrival_time):
    time_part_1 = arrival_time.split('T')[1]
    time_part_2 = time_part_1.split('-')[0]
    return time_part_2

arrival_time = "23:49:41"
print(arrival_time == "23:49:41" ) #time of arrival
#output should be in HH:MM:SS format like 23:49:31 for the last entry as above

## insert code to exact times of all arrivals on a day. Store in a list and␣
 ↪convert to series or dataframe called arrival_times
arrival_time_infos = []
for item in arrivals_info.iloc[1]:
    arrival_time = item["arrival"]
    arrival_time_infos.append(arrival_time)

arrival_times_list = []
for arrivals_time_info in arrival_time_infos:
    arrival_update_time = get_arrive_time(arrivals_time_info)
    arrival_times_list.append(arrival_update_time)

arrival_times = pd.DataFrame(arrival_times_list)
```

```
print(num_of_arrivals==arrival_times.shape[0]) #should evaluate to True
```

```
146
2024-07-01T23:49:41-05:00
True
True
```

[415]:
```
#So percent run is
print('Efficiency of the train system is:',len(arrivals_info.iloc[1])/
  ↪len(schedule_info.iloc[1]["allDepartures"]))
```

```
Efficiency of the train system is: 0.9358974358974359
```

1.6.2

Once you have figured out the data preprocessing to obtain actual arrival and scheduled times of the train from one json, you may write a function to extract them for all the 7 days (or json files), and then proceed to 1.2

1.6.3 Question 1.2 (4% credit)

Since the departure and arrival are given in HH:MM:SS format, we will write two helper functions to convert to minutes. Write two functions, `extract_hour` and `extract_mins` that converts this format to hours and minutes, respectively. Hint: You may want to use modular arithmetic and integer division. Keep in mind that the data has not been cleaned and you need to check whether the extracted values are valid. Replace all the invalid values with `NaN`. The documentation for `pandas.Series.where` provided here should be helpful.

[416]:
```
# 1% credit
### write code to test your extract_hour function here and execute it
# HINT: See tests_sample_part1/tests.py
def extract_hour(time):
    hour_list = []
    for t in time:
        if pd.isna(t):
            hour_list.append(np.nan)
        else:
            try:
                if t < 0 or t > 2359:
                    hour_list.append(np.nan)
                    continue

                time_int = int(t)
                hour = time_int // 100
                minute = time_int % 100

                if 0 <= hour < 24 and 0 <= minute < 60:
                    hour_list.append(hour)
                else:
```

```
                    hour_list.append(np.nan)
            except ValueError:
                hour_list.append(np.nan)
    return pd.Series(hour_list)

# Test example
ser = pd.Series([1030.0, 1259.0, np.nan, 2475,5521.0], dtype='float64')
extract_hour(ser)
# 0    10.0
# 1    12.0
# 2     NaN
# 3     NaN
# 4     NaN
# dtype: float64
```

[416]:
```
0    10.0
1    12.0
2     NaN
3     NaN
4     NaN
dtype: float64
```

[417]:
```
# 3% credit
def extract_mins(time):
    """
    Extracts minute information from military time

    Args:
        time (float64): series of time given in military format.
          Takes on values in 0.0-2359.0 due to float64 representation.

    Returns:
        array (float64): series of input dimension with minute information.
          Should only take on integer values in 0-59
    """
    minute_list = []
    for t in time:
        if pd.isna(t):
            minute_list.append(np.nan)
        else:
            try:
                if t < 0 or t > 2359:
                    minute_list.append(np.nan)
                    continue

                time_int = int(t)
                hour = time_int // 100
```

```
                minute = time_int % 100

                if 0 <= hour < 24 and 0 <= minute < 60:
                    minute_list.append(minute)
                else:
                    minute_list.append(np.nan)
            except ValueError:
                minute_list.append(np.nan)
    return pd.Series(minute_list)
```

[418]:
```
# 1% credit
### write code to test your extract_mins function here and execute it
# HINT: See tests_sample_part1/tests.py
ser = pd.Series([1030.0, 1259.0, np.nan, 2475], dtype='float64')
extract_mins(ser)
# 0    30.0
# 1    59.0
# 2     NaN
# 3     NaN
# dtype: float64
```

[418]:
```
0    30.0
1    59.0
2     NaN
3     NaN
dtype: float64
```

[419]:
```
# 2% credit
def convert_to_minofday(time):
    """
    Converts HH:MM time to minute of day

    Args:
        time: series of time given as strings in HH:MM:SS format.


    Returns:
        array (float64): series of input dimension with minute of day

    Example: 13:03 is converted to 783.0
    """
    min_of_day_list= []
    for t in time:
        try:
            if isinstance(t, str):
                parts = t.split(':')
```

```
                if len(parts) >= 2:
                    hours = float(parts[0])
                    minutes = float(parts[1])

                    if 0 <= hours < 24 and 0 <= minutes < 60:
                        min_of_day = hours * 60 + minutes
                    else:
                        min_of_day = np.nan
                else:
                    min_of_day = np.nan
            else:
                min_of_day = np.nan
        except (ValueError, AttributeError):
            min_of_day = np.nan

        min_of_day_list.append(min_of_day)
    return pd.Series(min_of_day_list)


# Test your code
ser = pd.Series(['13:03', '12:00', '24:00'])
convert_to_minofday(ser)
# 0    783.0
# 1    720.0
# 2      NaN
# dtype: float64
```

[419]:
```
0    783.0
1    720.0
2      NaN
dtype: float64
```

1.6.4 Question 1.3 (8% credit)

Before we can calculate delays, we need one more important function to write. Notice that arrival times and departure times are given as two lists or dataframes. Our first task is to relate them so that each entry in the arrivals has a corresponding scheduled times. We will assume that clearly num_of_arrivals <= num_scheduled as is the case in these files (check this yourself!)

[420]:
```
# 4%credit
def find_closest_scheduled(arrival, scheduled_times):
    min_diff = float('inf')
    closest_scheduled_time = None
    for scheduled_time in scheduled_times:
        diff = abs(arrival - scheduled_time)
        if diff < min_diff:
            min_diff = diff
```

```
            closest_scheduled_time = scheduled_time
    return closest_scheduled_time

def assigned_scheduled_times(arrival_times, scheduled_times):
    """
    Calculates delay times y - x

    Args:
        arrival_times: series of scheduled times
        scheduled_times: series of actual arrival times

    Returns:
        arrival_scheduled_times: pandas dataframe with two columns viz.,␣
 ↪arrival times and corresponding scheduled time
    """
    Update_scheduled_list = []
    for  arrival_time in arrival_times:
        closest_scheduled_time =␣
 ↪find_closest_scheduled(arrival_time,scheduled_times)
        Update_scheduled_list.append(closest_scheduled_time)

    arrival_scheduled_times = pd.DataFrame({
        'Arrival Times': arrival_times,
        'Scheduled Times': Update_scheduled_list,

    })
    return arrival_scheduled_times
```

[421]:
```
# 1% credit
def military_time_to_minutes(time):
    hour = extract_hour(time)
    minute = extract_mins(time)
    return hour * 60 + minute

def calc_delay(df_assigned_scheduled_times):
    """
    Calculates delay times y - x

    Args:
        df_assigned_scheduled_times: pandas dataframe with two columns viz.,␣
 ↪arrival times and corresponding scheduled time

    Returns:
        pandas series of input dimension with delay time
    """
    if df_assigned_scheduled_times.columns.isnull().all():
```

```
        df_assigned_scheduled_times.columns = ['Scheduled Times', 'Arrival␣
↪Times']
    if list(df_assigned_scheduled_times.columns) == [0, 1]:
        df_assigned_scheduled_times.columns = ['Scheduled Times', 'Arrival␣
↪Times']
    # Update_df_assigned_scheduled_times =␣
↪assigned_scheduled_times(df_assigned_scheduled_times['Arrival␣
↪Times'],df_assigned_scheduled_times['Scheduled Times'])

    delay_time = military_time_to_minutes(df_assigned_scheduled_times['Arrival␣
↪Times']) - military_time_to_minutes(df_assigned_scheduled_times['Scheduled␣
↪Times'])
    #delay_time = delay_time.clip(lower=0)

    return delay_time

#Test your code
sched = pd.Series([1303, 1210], dtype='float64')
actual = pd.Series([1304, 1215], dtype='float64')
df = assigned_scheduled_times(actual,sched)
print(df)
calc_delay(pd.concat([sched, actual], axis=1))
# 0    1.0
# 1    5.0
# dtype: float64
```

```
   Arrival Times  Scheduled Times
0         1304.0           1303.0
1         1215.0           1210.0
```

[421]:
```
0    1
1    5
dtype: int64
```

[422]:
```
# 3% credit
### write code to test your functions here by calculating delay between␣
↪`sched_dep_time` and `actual_dep_time` for each direction.
### your printed results should show the values of the following two variables

def get_delay_time(arrival_info_list, scheduled_info_list):
    arrive_time_list = []
    for item in arrival_info_list:
        arrive_time= item['arrival']
        # update time format
        update_arrive_time = get_arrive_time(arrive_time)[:5]
        arrive_time_list.append(update_arrive_time)
    all_arrive_time = pd.Series(arrive_time_list)
```

```
    arrive_time_min = convert_to_minofday(all_arrive_time)
    # find all scheduled_time
    scheduled_time_list = []
    for item in scheduled_info_list:
        # update time format
        update_scheduled_time = item[:5]
        scheduled_time_list.append(update_scheduled_time)
    all_scheduled_time = pd.Series(scheduled_time_list)
    scheduled_time_min = convert_to_minofday(all_scheduled_time)

    arrival_scheduled_time =␣
 ↪assigned_scheduled_times(arrive_time_min,scheduled_time_min)
    delay_result = arrival_scheduled_time['Arrival Times'] -␣
 ↪arrival_scheduled_time['Scheduled Times']

    return delay_result

delay_30111 = get_delay_time(arrivals_info.iloc[0],schedule_info.
 ↪iloc[0]["allDepartures"])
delay_30112 = get_delay_time(arrivals_info.iloc[1],schedule_info.
 ↪iloc[1]["allDepartures"])

delay = pd.concat([delay_30111, delay_30112], keys=['30111', '30112'])
print(delay)

condition = delay_30111 >= 20
result_indices_30111 = delay_30111.index[condition]

condition = delay_30112 >= 20
result_indices_30112 = delay_30112.index[condition]

Delay_times_20 =[]
for item in result_indices_30111:
    Delay_times_20.append(delay_30111[item])
for item in result_indices_30112:
    Delay_times_20.append(delay_30112[item])

train_id_20 =[]
for item in result_indices_30111:
    train_id_20.append(arrivals_info.iloc[0][item]["id"])
for item in result_indices_30112:
    train_id_20.append(arrivals_info.iloc[1][item]["id"])

delayed20 = pd.DataFrame({
        'Delay_times': Delay_times_20,
        'Train ID': train_id_20
    })
```

```
print (delayed20)
```

```
30111  0     -46.0
       1     -43.0
       2     -31.0
       3     -17.0
       4       1.0
              …
30112  141     2.0
       142     0.0
       143     6.0
       144    -9.0
       145    -9.0
Length: 287, dtype: float64
Empty DataFrame
Columns: [Delay_times, Train ID]
Index: []
```

1.6.5 Question 1.4 (10% credit)

Calculate the average delay for each day of the week and the weekly variance and submit

[423]:
```
#Calculate
#   1 (5%credit). the average delay for each day  -- 7 numbers for each␣
 ↪direction,
#   2 (5%credit). the variance between them the 7 numbers, and
#   3. store in a dataframe with 8 rows (last row for variance) and 2 columns,␣
 ↪and save in a csv file for submission

def get_one_day_average_delay(filepath):
    arrival_data = pd.read_json(filepath)
    arrivals_info = arrival_data.loc[:,'arrivals']
    schedule_info = arrival_data.loc[:,'scheduled']
    delay_30111 = get_delay_time(arrivals_info.iloc[0],schedule_info.
 ↪iloc[0]["allDepartures"])
    delay_30112 = get_delay_time(arrivals_info.iloc[1],schedule_info.
 ↪iloc[1]["allDepartures"])
    average_30111 = delay_30111.mean()
    average_30112 = delay_30112.mean()
    return average_30111,average_30112

d1_30111,d1_30112 = get_one_day_average_delay('20240701.json')
d2_30111,d2_30112 = get_one_day_average_delay('20240702.json')
d3_30111,d3_30112 = get_one_day_average_delay('20240703.json')
d4_30111,d4_30112 = get_one_day_average_delay('20240704.json')
d5_30111,d5_30112 = get_one_day_average_delay('20240705.json')
d6_30111,d6_30112 = get_one_day_average_delay('20240706.json')
```

```python
d7_30111,d7_30112 = get_one_day_average_delay('20240707.json')
avg_delay_30111 =␣
↪[d1_30111,d2_30111,d3_30111,d4_30111,d5_30111,d6_30111,d7_30111]
avg_delay_30112 =␣
↪[d1_30112,d2_30112,d3_30112,d4_30112,d5_30112,d6_30112,d7_30112]

df = pd.DataFrame({
    '30111 Direction': avg_delay_30111,
    '30112 Direction': avg_delay_30112
})

variance = df.var()
df.loc['Variance'] = variance

df.to_csv('average_delay_and_variance.csv')

print(df)
```

```
          30111 Direction  30112 Direction
0               -0.517730         0.068493
1               -0.114094         0.463087
2               -0.117241         0.387755
3               -0.445545         0.223301
4               -0.208633         0.569444
5               -1.056075         0.113043
6               -0.881818         0.452830
Variance         0.139167         0.036694
```

```
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
```

```
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
```

```
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
```

```
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
/var/folders/5p/qvx0rb6n1z32pm4tq8z0r8jm0000gn/T/ipykernel_39621/3603306433.py:7
: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings
is deprecated. In a future version, strings will be parsed as datetime strings,
matching the behavior without a 'unit'. To retain the old behavior, explicitly
cast ints or floats to numeric type before calling to_datetime.
  arrival_data = pd.read_json(filepath)
```

1.7 Part 2 (45% of HW 1): Web scraping and data collection

Here, you will practice collecting and processing data in Python. By the end of this exercise hopefully you should look at the wonderful world wide web without fear, comforted by the fact that anything you can see with your human eyes, a computer can see with its computer eyes. In particular, we aim to give you some familiarity with:

- Using HTTP to fetch the content of a website
- HTTP Requests (and lifecycle)
- RESTful APIs
  - Authentication (OAuth)
  - Pagination
  - Rate limiting
- JSON vs. HTML (and how to parse each)
- HTML traversal (CSS selectors)

Since everyone loves food (presumably), the ultimate end goal of this homework will be to acquire the data to answer some questions and hypotheses about the restaurant scene in Chicago (which we will get to later). We will download **both** the metadata on restaurants in Chicago from the Yelp API and with this metadata, retrieve the comments/reviews and ratings from users on restaurants.

1.7.1 Library Documentation

For solving this part, you need to look up online documentation for the Python packages you will use:

- Standard Library:
  - io
  - time
  - json
- Third Party
  - requests
  - Beautiful Soup (version 4)

1.8 Setup

First, import necessary libraries:

[424]:
```
import io, time, json
import requests
from bs4 import BeautifulSoup

import base64
```

1.9 Authentication and working with APIs

There are various authentication schemes that APIs use, listed here in relative order of complexity:

- No authentication
- HTTP basic authentication
- Cookie based user login
- OAuth (v1.0 & v2.0, see this post explaining the differences)
- API keys
- Custom Authentication

For the IMDb example below (**Q2.1**), since it is a publicly visible page we did not need to authenticate. HTTP basic authentication isn't too common for consumer sites/applications that have the concept of user accounts (like Facebook, LinkedIn, Twitter, etc.) but is simple to setup quickly and you often encounter it on with individual password protected pages/sites.

Cookie based user login is what the majority of services use when you login with a browser (i.e. username and password). Once you sign in to a service like Facebook, the response stores a cookie in your browser to remember that you have logged in (HTTP is stateless). Each subsequent request to the same domain (i.e. any page on `facebook.com`) also sends the cookie that contains the authentication information to remind Facebook's servers that you have already logged in.

Many REST APIs however use OAuth (authentication using tokens) which can be thought of a programmatic way to "login" *another* user. Using tokens, a user (or application) only needs to send the login credentials once in the initial authentication and as a response from the server gets a special signed token. This signed token is then sent in future requests to the server (in place of the user credentials).

A similar concept common used by many APIs is to assign API Keys to each client that needs access to server resources. The client must then pass the API Key along with *every* request it makes to the API to authenticate. This is because the server is typically relatively stateless and does not maintain a session between subsequent calls from the same client. Most APIs (including Spotify) allow you to pass the API Key via a special HTTP Header: `Authorization: Bearer <API_KEY>`. Check out the docs for more information.

1.9.1 Question 2.1: Basic HTTP Requests w/o authentication (5%)

First, let's do the "hello world" of making web requests with Python to get a sense for how to programmatically access web pages: an (unauthenticated) HTTP GET to download a web page.

Fill in the funtion to use `requests` to download and return the raw HTML content of the URL passed in as an argument. As an example try the following IMDb page to retrieve titles and descriptions of top 250 movies: `https://www.imdb.com/chart/top/`

Your function should return a string of: `<text>`.

(Hint: look at the **Library documentation** listed earlier to see how `requests` should work.)

[425]:
```
# 2% credit
def retrieve_html(url):
    """
    Return the raw HTML at the specified URL.

    Args:
        url (string):

    Returns:
        raw_html (string): the raw HTML content of the response, properly␣
 ↪encoded according to the HTTP headers.
    """
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/
 ↪537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36'
    }
    response = requests.get(url, headers=headers)
    html_content = response.text
    return html_content
```

[426]:
```
# Example usage
imdb_url = 'https://www.imdb.com/chart/top/'
imdb_data = retrieve_html(imdb_url)
print(imdb_data[:1000])
# <!DOCTYPE html><html lang="en-US" xmlns:og="http://opengraphprotocol.org/
 ↪schema/" xmlns:fb="http://www.facebook.com/2008/fbml"><head><meta␣
 ↪charSet="utf-8"/>...
```

```
<!DOCTYPE html><html lang="en-US"
xmlns:og="http://opengraphprotocol.org/schema/"
xmlns:fb="http://www.facebook.com/2008/fbml"><head><meta charSet="utf-8"/><meta
name="viewport" content="width=device-width"/><script>if(typeof uet ===
'function'){ uet('bb', 'LoadTitle', {wb: 1});
}</script><script>window.addEventListener('load', (event) => {
        if (typeof window.csa !== 'undefined' && typeof window.csa ===
'function') {
            var csaLatencyPlugin = window.csa('Content', {
                element: {
                    slotId: 'LoadTitle',
                    type: 'service-call'
                }
            });
            csaLatencyPlugin('mark', 'clickToBodyBegin', 1726782623976);
        }
    })</script><title>IMDb Top 250 Movies</title><meta name="description"
content="As rated by regular IMDb voters." data-id="main"/><script type="applica
```

```
tion/ld+json">{"@type":"ItemList","itemListElement":[{"@type":"ListItem","item":
{"@type":"Movie","url":"https://www.imdb.com/title/tt
```

[427]:
```
#3% credit
def parse_imdb(imdb_data):
    """
    Return the movie lists from imdb top chart URL.

    Args:
        raw_html (string):

    Returns:
        movies (list): the list of movies with Title, Description and Rating.

        Example:
        movies = [
        {
            'Title': 'The Shawshank Redemption',
            'Description': 'A Maine banker convicted of the murder of his wife␣
 ↪and her lover...',
            'Rating': 9.3,
        },
        {
            'Title': 'The Godfather',
            'Description': 'Don Vito Corleone, head of a mafia family, decides␣
 ↪to hand over his empire...',
            'Rating': 9.2,

        },
            # ... more
        ]
    """
    soup = BeautifulSoup(imdb_data, 'html.parser')
    json_ld_script = soup.find('script', type='application/ld+json')
    json_data = json_ld_script.string
    data = json.loads(json_data)
    movies = []
    for item in data['itemListElement']:
        movie = item['item']
        title = movie['name']
        description = movie['description']
        aggregateRating = movie['aggregateRating']
        rating = aggregateRating['ratingValue']
        movies.append({
            'Title': title,
            'Description': description,
            'Rating' : rating
```

```
        })

    return  movies
```

[428]:
```
# Example usage
if imdb_data:
    movies = parse_imdb(imdb_data)
    for movie in movies[:3]:
        print(f"Title: {movie['Title']}")
        print(f"Description: {movie['Description']}")
        print(f"Rating: {movie['Rating']}")
else:
    print("Failed to retrieve the webpage content.")

# Example outputs
# Title: The Shawshank Redemption
# Description: A Maine banker convicted of the murder of his wife and her lover␣
 ↪in 1947 gradually forms a friendship over a quarter century with a hardened␣
 ↪convict, while maintaining his innocence and trying to remain hopeful␣
 ↪through simple comp...
# Rating: 9.3
# Title: The Godfather
# Description: Don Vito Corleone, head of a mafia family, decides to hand over␣
 ↪his empire to his youngest son, Michael. However, his decision␣
 ↪unintentionally puts the lives of his loved ones in grave danger.
# Rating: 9.2
# Title: The Dark Knight
# Description: When the menace known as the Joker wreaks havoc and chaos on the␣
 ↪people of Gotham, Batman, James Gordon and Harvey Dent must work together to␣
 ↪put an end to the madness.
# Rating: 9
```

```
Title: The Shawshank Redemption
Description: A banker convicted of uxoricide forms a friendship over a quarter
century with a hardened convict, while maintaining his innocence and trying to
remain hopeful through simple compassion.
Rating: 9.3
Title: The Godfather
Description: The aging patriarch of an organized crime dynasty transfers control
of his clandestine empire to his reluctant son.
Rating: 9.2
Title: The Dark Knight
Description: When a menace known as the Joker wreaks havoc and chaos on the
people of Gotham, Batman, James Gordon and Harvey Dent must work together to put
an end to the madness.
Rating: 9
```

Now while this example might have been fun, we haven't yet done anything more than we could

with a web browser. To really see the power of programmatically making web requests we will need to interact with an API. For the rest of this lab we will be working with the Spotify Web API.

1.10 Spotify Web API Access

The reasons for using the Spotify Web API are threefold:

1. Incredibly Rich Dataset:
   - Track and Artist Data: detailed information about tracks and artists.
   - Genre and Popularity Trends: Analyze trends in music genres and track popularity over time.
   - Audio Features: Analyze tracks based on audio features like tempo, key, and more.
   - Personal Relevance: the Spotify API enables you to find data that resonates with your personal interests and preferences, making the analysis more engaging and insightful.
2. Well-Documented API: The Spotify Web API Documentation provides thorough examples and guides to help you get started with API requests, handling responses, and understanding the rich dataset available through Spotify.

To access the Spotify API, you will need to perform a few steps. Like many other platforms, Spotify uses authentication and rate limiting to control access to its data. This ensures fair usage and compliance with Spotify's policies. The first step (even before making any request) is to set up a Spotify Developer account and obtain API credentials.

1. Create a Spotify account (if you do not have one already)
2. Log into the dashboard using your Spotify account.
3. Generate API keys (if you haven't already). Create an app and select "Web API" for the question asking which APIs are you planning to use. Once you have created your app, you will have access to the app credentials. These will be required for API authorization to obtain an access token.

Now that we have our accounts setup we can start making requests!

1.10.1 Question 2.2: Authenticated HTTP Request with the Spotify Web API (12%)

First, store your Spotify credentials in a local file (kept out of version control) which you can read in to authenticate with the API. This file can be any format/structure since you will fill in the function stub below.

For example, you may want to store your key in a file called `spotify_api_key.json` (run in terminal):

```
echo '{"client_id": "your_client_id", "client_secret": "your_client_secret_key"}' > spotify_api
```

KEEP THE API KEY FILE PRIVATE AND OUT OF VERSION CONTROL (and definitely do not submit them to Gradescope!)

You can then read from the file using:

[429]:
```
# 1% credit
with open('spotify_api_key.json', 'r') as file:
    credentials = json.load(file)
    client_id = credentials['client_id']
```

```
    client_secret = credentials['client_secret']
    print(client_id, client_secret)
    # verify your credentials are correct
# DO NOT FORGET TO CLEAR THE OUTPUT TO KEEP YOUR API KEY PRIVATE
```

```
b0e357c92a534fbbb8c8fa3a3c924213 28bbb0f189c04564b41852f71eaaaee1
```

[430]:
```
# 1% credit
def read_api_key(filepath):
    """
    Read the Spotify API Keys from file.

    Args:
        filepath (string): File containing API Keys
    Returns:
        client_id (string): Your client id
        client_secret (string): Your client secret
    """

    # feel free to modify this function if you are storing the API Key␣
 ↪differently
    with open(filepath, 'r') as file:
       credentials = json.load(file)
    client_id = credentials['client_id']
    client_secret = credentials['client_secret']
    return client_id, client_secret
    ''' return json.load(file)'''
```

Now authenticate and get access token for future search.

Hint: read Authorization Code Flow docs. 1. Send a POST request to the /api/token endpoint. 2. Parse the response json to get access_token

[431]:
```
# 2% credit
def access_spotify(client_id, client_secret):
    """
    Authenticates the user and retrieves the bearer token required for API␣
 ↪requests.
    """
    #
    auth_url = 'https://accounts.spotify.com/api/token'
    auth_string = f"{client_id}:{client_secret}"

    auth_bytes = auth_string.encode('ascii')
    auth_base64 = base64.b64encode(auth_bytes).decode('ascii')
    auth_header = auth_base64

    headers = {
```

```
        'Authorization': f'Basic {auth_header}',
        'user-agent':'my-app/0.01'
    }
    data = {
        'grant_type': 'client_credentials'
    }

    response = requests.post(auth_url, headers=headers, data=data)
    token_info = response.json()
    access_token = token_info['access_token']
    return access_token


# Example usage
# DO NOT FORGET TO CLEAR THE OUTPUT TO KEEP YOUR API KEY PRIVATE

access_token = access_spotify(client_id, client_secret)
print(f"Authenticated with token: {access_token}")
```

```
Authenticated with token: BQDSFAzjf1ZWlwmNoimFguPEreILCGzi-FHSLMXZQNnnaSO5hoKjWv
8BbOP19STizzeEPKoWfrTXQzBIW49KnUH3r-GUN6HsbPZF7nQ8-LcamOc_huk
```

Using the Spotify API, fill in the following function stub to make an authenticated request to the Search Endpoint. Once you have the access token, you can use it to search for content (Tracks, Artists, or Albums) on Spotify.

[432]:
```
# 4% credit
def spotify_search_params(client_id, client_secret, **kwargs):
    """
    Construct url, headers and params. Reference API docs (link above) to use␣
 ↪the arguments
    """
    # What is the url endpoint for search?
    url = "https://api.spotify.com/v1/search"
    # How is Authentication performed? Hint: use access_token from function of␣
 ↪access_spotify
    access_token = access_spotify(client_id, client_secret)
    headers = headers = {
        'Authorization': f'Bearer {access_token}',
        'Content-Type': 'application/json',
    }
    query_params = []
    for key, value in kwargs.items():
        if key in ['artist', 'track', 'album']:
            query_params.append(f'{key}:{value}')
    query = ' '.join(query_params)

    # Include keyword arguments in params dictionary
```

```
    params = {
        'q': query,
        'type': kwargs.get("type", "track"),
        'limit': kwargs.get("limit", 10),
        'offset': kwargs.get("offset", 0),
    }

    return url, headers, params
```

Hint: `**kwargs` represent keyword arguments that are passed to the function. For example, if you called the function `spotify_search_params(client_id, client_secret, artist="Taylor Swift", track="Love", type="track", limit=10, offset=0))`. The arguments `client_id` and `client_secret` are called *positional arguments* and key-value pair arguments are called **keyword arguments**. Your `kwargs` variable will be a python dictionary with those keyword arguments.

[433]:
```
# Example usage
url, headers, params = spotify_search_params(
    client_id, client_secret,
    artist="Taylor Swift",
    track="Lover",
    type="track",
    limit=5,
    offset=0
)
url, headers, params
# ('https://<hidden_url_check_search_endpoint_docs_to_get_answer>',
#  {'Authorization': 'Bearer your_access_token'},
#  {'q': 'artist:Taylor Swift track:Love', 'type': 'track', 'limit': 5,'offset':
 ↪0})
```

[433]:
```
('https://api.spotify.com/v1/search',
 {'Authorization': 'Bearer BQDqxvSulfdaTsoLxwD31cISeKgXf5yq6eTiM--V1xhSh5zkvV-
xCeHqa8cjDGieOKC4VjU1R-wc8wkxYhDLeJEHsyBNd_5H9kftkcH8AXORiyjgLag',
  'Content-Type': 'application/json'},
 {'q': 'artist:Taylor Swift track:Lover',
  'type': 'track',
  'limit': 5,
  'offset': 0})
```

Now use `spotify_search_params(client_id, client_secret, **kwargs)` to actually search album/track/artist from Spotify API. Most of the code is provided to you.

[434]:
```
# 2% credit
def api_get_request(url, headers, params):
    """
    Send a HTTP GET request and return a json response
```

```
    Args:
        url (string): API endpoint url
        headers (dict): A python dictionary containing HTTP headers including␣
↪Authentication to be sent
        url_params (dict): The parameters (required and optional) supported by␣
↪endpoint

    Returns:
        results (json): response as json
    """
    # See requests.request?
    response = requests.get(url, headers=headers, params=params)
    return response.json()


def spotify_search(client_id, client_secret, **kwargs):
    """
    Make an authenticated request to the Spotify API and return search results.

    Args:
        client_id (string): Your Spotify Client ID for Authentication
        client_secret (string): Your Spotify Client Secret for Authentication
        **kwargs: Additional search parameters (e.g., artist, track, album, etc.
↪)

    Returns:
        total (integer): Total number of tracks matching the query
        tracks (list): List of dicts representing each track with name, and␣
↪popularity
    """
    url, headers, params = spotify_search_params(client_id, client_secret,␣
↪**kwargs)
    response_json = api_get_request(url, headers, params)
    total = response_json['tracks']['total']
    tracks = []
    if response_json['tracks']['items']:
            popularities = []
            for track in response_json['tracks']['items']:
                track_info = {
                    'track_name': track['name'],
                    'popularity': track['popularity']
                }
                tracks.append(track_info)
                popularities.append(track['popularity'])

    return total, tracks
```

```
# 2% credit
client_id, client_secret = read_api_key('spotify_api_key.json')
total, tracks = spotify_search(client_id, client_secret,artist="Taylor Swift",␣
 ↪track="Lover", type="track", limit=5)
print(total)
#44
print(tracks)
#[{'track_name': 'Lover', 'album_name': 'Lover', 'popularity': 86},␣
 ↪{'track_name': 'Lover (Remix) [feat. Shawn Mendes]', 'album_name': 'Lover␣
 ↪(Remix) [feat. Shawn Mendes]', 'popularity': 67}, {'track_name': 'Lover -␣
 ↪First Dance Remix', 'album_name': 'Lover (First Dance Remix)', 'popularity':␣
 ↪53}, {'track_name': 'Lover - Live From Paris', 'album_name': 'Lover (Live␣
 ↪From Paris)', 'popularity': 48}, {'track_name': 'Lover', 'album_name':␣
 ↪'Sunset Vibes', 'popularity': 16}]
```

```
44
[{'track_name': 'Lover', 'popularity': 86}, {'track_name': 'Lover (Remix) [feat.
Shawn Mendes]', 'popularity': 67}, {'track_name': 'Lover - First Dance Remix',
'popularity': 53}, {'track_name': 'Lover - Live From Paris', 'popularity': 48},
{'track_name': 'Lover', 'popularity': 16}]
```

Now that we have completed the "hello world" of working with the Spotify API, we are ready to really fly! The rest of the exercise will have a bit less direction since there are a variety of ways to retrieve the requested information but you should have all the component knowledge at this point to work with the API.

1.11 Parameterization and Pagination

Before we can retrieve all of Taylor Swift's tracks, albums, or playlists, we need to understand how to work with the Spotify API's search and pagination system. Spotify's API returns a limited number of results per request to safeguard against returning TOO much data at once (imagine if you were to request 100,000 tracks in one go!). This limitation is common among APIs and helps manage rate limiting while ensuring efficient and fair access to Spotify's vast music database.

> As a thought exercise, consider: If an API has 1,000,000 records, but only returns 10 records per page and limits you to 5 requests per second… how long will it take to acquire ALL of the records contained in the API?

One of the ways that APIs are an improvement over plain web scraping is the ability to make **parameterized** requests. Just like the Python functions you have been writing have arguments (or parameters) that allow you to customize its behavior/actions (an output) without having to rewrite the function entirely, we can parameterize the queries we make to the Spotify API to filter the results it returns.

1.11.1 Question 2.3: Retrieve All Tracks by Taylor Swift on Spotify (10%)

Again using the API documentation for the `search` endpoint, fill in the following function to retrieve all of the *Tracks* for a given query. Again you should use your `read_api_key()` function to read the API Key used for the requests. You will need to account for **pagination** and **rate limiting** to:

1. Retrieve all of the Track objects (# of track objects should equal `total` in the response). **Paginate by querying 10 restaurants each request.**
2. Pause slightly (at least 200 milliseconds) between subsequent requests so as to not overwhelm the API (and get blocked).

As always with API access, make sure you follow all of the API's policies and use the API responsibly and respectfully.

DO NOT MAKE TOO MANY REQUESTS TOO QUICKLY OR YOUR KEY MAY BE BLOCKED

[435]:
```
# 4% credit
def paginated_spotify_search_requests(client_id, client_secret, artist_name,␣
 ↪total,limit):
    """
    Returns a list of tuples (url, headers, params) for paginated search of all␣
 ↪restaurants
    Args:
        client_id, client_secret (string): Your Spotify API Key for␣
 ↪Authentication
        artist_name (string): Artist name
        total (int): Total number of items to be fetched
        limit (int): Number of items to fetch per request (default is 50)
    Returns:
        results (list): list of tuple (url, headers, params)
    """
    # HINT: Use total, offset and limit for pagination
    # You can reuse function location_search_params(...)
    results = []
    num_pages = (total + limit - 1) // limit
    for page in range(num_pages):
        offset = page * limit
        url, headers, params = spotify_search_params(client_id,␣
 ↪client_secret,artist=artist_name,limit=limit,offset=offset)
        results.append((url, headers, params))

    return results

# Example Usage
client_id, client_secret = read_api_key('spotify_api_key.json')
artist_name = "Taylor Swift"
total=200
limit=50
all_track_requests = paginated_spotify_search_requests(client_id,␣
 ↪client_secret, artist_name, total,limit)
all_track_requests

#[('https:<hidden>',
```

```
#  {'Authorization': 'Bearer your_access_token'},
#  {'q': 'artist:Taylor Swift', 'type': 'track', 'limit': 50, 'offset': 0}),
# ('https:<hidden>',
#  {'Authorization': 'Bearer your_access_token'},
#  {'q': 'artist:Taylor Swift', 'type': 'track', 'limit': 50, 'offset': 50}),
# ('https:<hidden>',
#  {'Authorization': 'Bearer your_access_token'},
#  {'q': 'artist:Taylor Swift', 'type': 'track', 'limit': 50, 'offset': 100}),
# ('https:<hidden>',
#  {'Authorization': 'Bearer your_access_token'},
#  {'q': 'artist:Taylor Swift', 'type': 'track', 'limit': 50, 'offset': 150})]
```

[435]:
```
[('https://api.spotify.com/v1/search',
  {'Authorization': 'Bearer BQDjuaYx_76uqD_fWy1kDcfQBC4YRqDflsbV2ATobfL2FkkvjdvF
GjdX3l903n66ltTrwmD8t6a37XDVmgyIx4sp4HiOh5_iBOsuWSAyBOVNt9Y-Dgo',
   'Content-Type': 'application/json'},
  {'q': 'artist:Taylor Swift', 'type': 'track', 'limit': 50, 'offset': 0}),
 ('https://api.spotify.com/v1/search',
  {'Authorization': 'Bearer BQCjRDtttvTl-
9lJ7t9IcPioyivkQ1iCHO8xGbRscMwhUTD77yXs4tfQZpYB0k78BgMs02DIGPa9jpSNkqSVUP-
FbXJrUvKh4ofUbhXklDyS0qn8DyM',
   'Content-Type': 'application/json'},
  {'q': 'artist:Taylor Swift', 'type': 'track', 'limit': 50, 'offset': 50}),
 ('https://api.spotify.com/v1/search',
  {'Authorization': 'Bearer BQD7_PdhqciajYmreqoWP5TlMqM9fN8CS5BjRuj7SLZtQHTLOKj9
cCjKxWdcOlDefUiRSnNTgIGk8RiVj4cRKImSWmdmNR8Avj9ncPlj6wmjpwQqK5A',
   'Content-Type': 'application/json'},
  {'q': 'artist:Taylor Swift', 'type': 'track', 'limit': 50, 'offset': 100}),
 ('https://api.spotify.com/v1/search',
  {'Authorization': 'Bearer BQDrI7Ea5ZpQtXiHv-
6Z6O92gKGRjAnaLMkYVbbMfenfHB5fHIJQaQzabDl3lUnTemrW_QWWvfndECU1KK5PiISAFeFfI_XRT-
Vholb854NumrxaIP0',
   'Content-Type': 'application/json'},
  {'q': 'artist:Taylor Swift', 'type': 'track', 'limit': 50, 'offset': 150})]
```

[436]:
```
# 3% credit
def get_tracks(client_id, client_secret, artist_name):
    """
    Construct the pagination requests for ALL tracks by Given Artist on Spotify.

    Args:
        client_id (string): Your Spotify Client ID for Authentication
        client_secret (string): Your Spotify Client Secret for Authentication
        artist_name (string): Artist name

    Returns:
        results (list): List of dicts representing each track
```

```
    """
    total_items = 300
    limit = 50

    tracks_request = paginated_spotify_search_requests(client_id,␣
 ↪client_secret,artist_name, total_items,limit)

    # Use returned list of (url, headers, url_params) and function␣
 ↪api_get_request to retrive all restaurants
    # REMEMBER to pause slightly after each request.
    result = []

    for url, headers, params in tracks_request:
        response_json = api_get_request(url, headers, params)
        if 'tracks' in response_json and 'items' in response_json['tracks']:
            for track in response_json['tracks']['items']:
                album = track['album']
                track_info = {
                    'track_name': track['name'],
                    'album_name': album['name'],
                    'popularity': track['popularity']
                }
                result.append(track_info)
    return result
```

[437]:
```
# 3% credit
artist_name = 'Taylor Swift'
data = get_tracks(client_id, client_secret, artist_name)
print(len(data))
# 300

# Display first 10 tracks with Track name, Album name and Popularity
for track in data[:10]:
    track_name = track.get('track_name', 'Unknown')
    album_name = track.get('album_name', 'Unknown')
    popularity = track.get('popularity', 'Unknown')
    print(f'Track Name: {track_name}, Album: {album_name}, Popularity:␣
 ↪{popularity}')
#Track: Cruel Summer, Album: Lover, Popularity: 92
#Track: Fortnight (feat. Post Malone), Album: THE TORTURED POETS DEPARTMENT,␣
 ↪Popularity: 89
#Track: I Can Do It With a Broken Heart, Album: THE TORTURED POETS DEPARTMENT,␣
 ↪Popularity: 86
#Track: august, Album: folklore, Popularity: 87
#Track: us. (feat. Taylor Swift), Album: The Secret of Us, Popularity: 82
#Track: Lover, Album: Lover, Popularity: 86
#Track: Don't Blame Me, Album: reputation, Popularity: 85
```

```
#Track: Anti-Hero, Album: Midnights, Popularity: 84
#Track: cardigan, Album: folklore, Popularity: 85
#Track: ...Ready For It?, Album: reputation, Popularity: 81
```

```
300
Track Name: Cruel Summer, Album: Lover, Popularity: 92
Track Name: Fortnight (feat. Post Malone), Album: THE TORTURED POETS DEPARTMENT,
Popularity: 88
Track Name: I Can Do It With a Broken Heart - Dombresky Remix, Album: I Can Do
It With a Broken Heart - Dombresky Remix, Popularity: 63
Track Name: I Can Do It With a Broken Heart, Album: THE TORTURED POETS
DEPARTMENT, Popularity: 86
Track Name: I Can Do It With a Broken Heart, Album: I Can Do It With a Broken
Heart - Dombresky Remix, Popularity: 55
Track Name: I Can Do It With a Broken Heart - Instrumental, Album: I Can Do It
With a Broken Heart - Dombresky Remix, Popularity: 52
Track Name: I Can Do It With a Broken Heart - Dombresky Remix, Album: I Can Do
It With a Broken Heart - Dombresky Remix, Popularity: 38
Track Name: I Can Do It With a Broken Heart, Album: I Can Do It With a Broken
Heart - Dombresky Remix, Popularity: 30
Track Name: I Can Do It With a Broken Heart - Instrumental, Album: I Can Do It
With a Broken Heart - Dombresky Remix, Popularity: 25
Track Name: us. (feat. Taylor Swift), Album: The Secret of Us, Popularity: 82
```

Now that we have the metadata on 300 tracks of Taylor Swift on Spotify, we can retrieve the album cover images. For that we need to download from image links, but to find out what pages to download we first need to parse our JSON from the API to extract the URLs of the tracks.

In general, it is a best practice to separate the act of **downloading** data and **parsing** data. This ensures that your data processing pipeline is modular and extensible (and autogradable ;). This decoupling also solves the problem of expensive downloading but cheap parsing (in terms of computation and time).

1.11.2 Question 2.4: Parse the API Responses and Extract the URLs (7%)

Because we want to separate the **downloading** from the **parsing**, fill in the following function to parse the URLs pointing to the tracks. As input your function should expect a string of properly formatted JSON (which is similar to **BUT** not the same as a Python dictionary) and as output should return a Python list of strings. Hint: print your `data` to see the JSON-formatted information you have. The input JSON will be structured as follows (same as the sample on the Spotify API page):

```
{
  "album": {
    "album_type": "compilation",
    "total_tracks": 9,
    "available_markets": ["CA", "BR", "IT"],
    "external_urls": {
      "spotify": "string"
    },
```

```
    "href": "string",
    "id": "2up3OPMp9Tb4dAKM2erWXQ",
    "images": [
      {
        "url": "https://i.scdn.co/image/ab67616d00001e02ff9ca10b55ce82ae553c8228",
        "height": 300,
        "width": 300
      }
    ],
    "name": "string",
    "release_date": "1981-12",
    "release_date_precision": "year",
    "restrictions": {
      "reason": "market"
    },
    "type": "album",
    "uri": "spotify:album:2up3OPMp9Tb4dAKM2erWXQ",
    "artists": [
      {
        "external_urls": {
          "spotify": "string"
        },
        "href": "string",
        "id": "string",
        "name": "string",
        "type": "artist",
        "uri": "string"
      }
    ]
  },
  "artists": [
    {
      "external_urls": {
        "spotify": "string"
      },
      "href": "string",
      "id": "string",
      "name": "string",
      "type": "artist",
      "uri": "string"
    }
  ],
  "available_markets": ["string"],
  "disc_number": 0,
  "duration_ms": 0,
  "explicit": false,
  "external_ids": {
    "isrc": "string",
```

```
    "ean": "string",
    "upc": "string"
  },
  "external_urls": {
    "spotify": "string"
  },
  "href": "string",
  "id": "string",
  "is_playable": false,
  "linked_from": {
  },
  "restrictions": {
    "reason": "string"
  },
  "name": "string",
  "popularity": 0,
  "preview_url": "string",
  "track_number": 0,
  "type": "track",
  "uri": "string",
  "is_local": false
}
```


[438]:
```python
# 4% credit
def parse_api_response(data):
    """
    Parse Spotify API results to extract cover images URLs.

    Args:
        data (string): String of properly formatted JSON.

    Returns:
        (list): list of URLs as strings from the input JSON.
    """
    all_image_urls = []

    for track in data['tracks']['items']:
        if 'album' in track and 'images' in track['album']:
            for image in track['album']['images']:
                all_image_urls.append(image['url'])

    return all_image_urls

# 3% credit
url, headers, params = spotify_search_params(
    client_id, client_secret,
    artist="Taylor Swift",
```

```
    track="Lover",
    type="track",
    limit=2,
    offset=0
)
response = requests.get(url, headers=headers, params=params)
response_text = response.json()
parse_api_response(response_text)

#['https://i.scdn.co/image/ab67616d0000b273e787cffec20aa2a396a61647',
# 'https://i.scdn.co/image/ab67616d00001e02e787cffec20aa2a396a61647',
# 'https://i.scdn.co/image/ab67616d00004851e787cffec20aa2a396a61647',
# 'https://i.scdn.co/image/ab67616d0000b27359457bdb1edb5c6417f3baa2',
# 'https://i.scdn.co/image/ab67616d00001e0259457bdb1edb5c6417f3baa2',
# 'https://i.scdn.co/image/ab67616d0000485159457bdb1edb5c6417f3baa2']
```

[438]:
```
['https://i.scdn.co/image/ab67616d0000b273e787cffec20aa2a396a61647',
 'https://i.scdn.co/image/ab67616d00001e02e787cffec20aa2a396a61647',
 'https://i.scdn.co/image/ab67616d00004851e787cffec20aa2a396a61647',
 'https://i.scdn.co/image/ab67616d0000b27359457bdb1edb5c6417f3baa2',
 'https://i.scdn.co/image/ab67616d00001e0259457bdb1edb5c6417f3baa2',
 'https://i.scdn.co/image/ab67616d0000485159457bdb1edb5c6417f3baa2']
```

As we can see, JSON is quite trivial to parse (which is not the case with HTML as we will see in a second) and work with programmatically. This is why it is one of the most ubiquitous data serialization formats (especially for ReSTful APIs) and a huge benefit of working with a well defined API if one exists. But APIs do not always exists or provide the data we might need, and as a last resort we can always scrape web pages…

1.12 Working with Web Pages (and HTML)

Think of APIs as similar to accessing an application's database itself (something you can interactively query and receive structured data back). But the results are usually in a somewhat raw form with no formatting or visual representation (like the results from a database query). This is a benefit *AND* a drawback depending on the end use case. For data science and *programatic* analysis this raw form is quite ideal, but for an end user requesting information from a *graphical interface* (like a web browser) this is very far from ideal since it takes some cognitive overhead to interpret the raw information. And vice versa, if we have HTML it is quite easy for a human to visually interpret it, but to try to perform some type of programmatic analysis we first need to parse the HTML into a more structured form.

> As a general rule of thumb, if the data you need can be accessed or retrieved in a structured form (either from a bulk download or API) prefer that first. But if the data you want (and need) is not as in our case we need to resort to alternative (messier) means.

We may like to scrape more information of tracks, such as lyrics. However, due to Spotify's API's policy, we are not able to scrape track pages. Always ensure you're using APIs and accessing data legally and ethically, respecting the terms of service of the platform you're working with.

Going back to the "hello world" example of question 2.1 with the AP News, we will retrieve the HTML of the movie site to get more interesting information as text, such as storyline and reviews.

1.12.1 Question 2.5: Parse a Movie Page from imdb (4%)

Using `BeautifulSoup`, parse the HTML of a single movie page to extract the reviews in a structured form as well as the URL to the next page of reviews (or `None` if it is the last page). Fill in following function stubs to parse a single page of reviews and return: * the reviews as a structured Python dictionary * the HTML element containing the link/url for the next page of reviews (or None).

For each review be sure to structure your Python dictionary as follows (to be graded correctly). The order of the keys doesn't matter, only the keys and the data type of the values:

```
{
    'Author': str
    'Rating': float
    'Date': str ('dd mm yyyy')
    'Review': str
}

# Example
{
    'Author': 'ap_griffiths'
    'Rating': 5
    'Date': '24 January 2012'
    'Review': "This is not a bad film per se, had it been about 30-40 minutes shorter I would
}
```

There can be issues with Beautiful Soup using various parsers, for maximum compatibility (and fewest errors) initialize the library with the default (and Python standard library parser): `BeautifulSoup(markup, "html.parser")`.

Most of the function has been provided to you:

[439]:
```python
# 4% credit
def fetch_html(url):
    """
    Fetch the HTML content of the specified URL.

    Args:
        url (string): The URL of the IMDb page.

    Returns:
        response_text (string): The raw HTML content of the page.
    """
    response = requests.get(url)
    if response.status_code == 200:
        return response.text
    else:
        raise Exception(f"Failed to fetch page: {response.status_code}")
```

```
def html_fetcher(url):
    """
    Fetch the HTML content and status code from the specified URL.

    Args:
        url (string): The URL of the webpage.

    Returns:
        (int, string): A tuple containing the status code and the raw HTML␣
 ↪content of the page.
    """
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/
 ↪537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.3'}
    response = requests.get(url, headers=headers)
    return response.status_code, response.text


def parse_page(html):
    """
    Parse reviews from an IMDb movie reviews page.

    Args:
        html (string): HTML content of the IMDb reviews page.

    Returns:
        reviews (list): A list of dictionaries, each containing the review's␣
 ↪rating, author, date, and content.
    """
    soup = BeautifulSoup(html,'html.parser')
    reviews_list = []

    # Find all review containers on the page
    review_containers = soup.find_all('div', class_='lister-item-content')
    # HINT: print reviews to see what http tag to extract

    for review_container in review_containers:
        rating_tag = review_container.find('span',␣
 ↪class_='rating-other-user-rating')
        rating = int(rating_tag.find('span').text) if rating_tag else None
        author = review_container.find('span', class_='display-name-link').text.
 ↪strip()
        review_date = review_container.find('span', class_='review-date').text.
 ↪strip()
        content = review_container.find('div', class_='text').text.strip()
```

```
        review = {
            'rating': rating,
            'Author': author,
            'review_date': review_date,
            'review_content': content
        }
        reviews_list.append(review)

    return reviews_list

# Example Usage
code, html = html_fetcher("https://www.imdb.com/title/tt1375666/reviews?
 ↪ref_=tt_urv") #should load inception movie released in 2010
reviews_list = parse_page(html)
print(len(reviews_list)) # 25
```

25

1.12.2 Question 2.6: Extract all IMDb reviews for a Single Movie (7%)

So now that we have parsed the first 25 reviews, and figured out a method to actually crawl through web pages! However, we noticed that Inception has over 4,000 reviews. IMDb uses a "load more" button that dynamically loads more reviews via AJAX requests when clicked.

Using the provided `html_fetcher` (for a real use-case you would use `requests`), programmatically retrieve **ALL** of the reviews for a **single** movie (provided as a parameter). Just like the API was paginated, the HTML paginates its reviews (it would be a very long web page to show 300 reviews on a single page) and to get all the reviews you will need to parse and traverse the HTML. As input your function will receive a URL corresponding to a Yelp restaurant. As output return a list of dictionaries (structured the same as question 2.5) containing the relevant information from the reviews. You can use `parse_page()` here.

[440]:
```
# 4% credits
def get_ajax_url_and_key(html):
    soup = BeautifulSoup(html, 'html.parser')
    load_more_button = soup.find('div', class_='load-more-data')

    if load_more_button:
        ajax_url = "https://www.imdb.com" + load_more_button['data-ajaxurl']
        pagination_key = load_more_button['data-key']
        return ajax_url, pagination_key
    else:
        return None, None

def fetch_ajax_page(ajax_url, pagination_key=None):
    headers = {
```

```
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/
↪537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.3'
    }
    params = {'ref_': 'undefined'}
    if pagination_key:
        params['paginationKey'] = pagination_key
    response = requests.get(ajax_url, headers=headers, params=params)

    if response.status_code == 200:
        soup = BeautifulSoup(response.text, 'html.parser')
        new_pagination_key = soup.find('div', {'data-key': True})
        new_pagination_key = new_pagination_key['data-key'] if␣
↪new_pagination_key else None
        return new_pagination_key, response.text
    else:
        raise Exception(f"Failed to fetch AJAX page: {response.status_code}")

def all_reviews(url):
    """
    Retrieve ALL of the reviews for a single restaurant on Yelp.

    Parameters:
        url (string): Yelp URL corresponding to the restaurant of interest.
        html_fetcher (function): A function that takes url and returns html␣
↪status code and content

    Returns:
        reviews (list): list of dictionaries containing extracted review␣
↪information
    """
    all_reviews = []
    code, initial_html = html_fetcher(url)
    ajax_url, pagination_key = get_ajax_url_and_key(initial_html)

    '''if not ajax_url or not pagination_key:
        raise Exception("Failed to find the AJAX URL or pagination key for␣
↪loading more reviews.")'''

    while True:
        pagination_key, html = fetch_ajax_page(ajax_url, pagination_key)
        reviews = parse_page(html)
        all_reviews.extend(reviews)

        if not pagination_key:
            break

    return all_reviews
```

You can test your function with this code:

[441]:
```
# 3% credits
data = all_reviews('https://www.imdb.com/title/tt1375666/reviews?ref_=tt_urv')
print(len(data))
# 4881
print(data[1])
# {'rating': '9', 'author': 'clipturnity', 'date': '22 August 2010',␣
 ↪'review_text': "I'd like to keep my review rather to the point.Pros: 1. its␣
 ↪theme - dream is a fascinating topic to say the least. There are a lot of␣
 ↪unknowns in dreamworld.2. its plot - there are several sweet twists and␣
 ↪unpredictable turns.3. its edgy drive - although you know what's coming␣
 ↪next, still you feel jumpy about it when it does.4. its rapid storyline -␣
 ↪the story moves fast from one scene to another, making the viewers feel like␣
 ↪on a roller coaster ride. At times, it's hard to keep up, even after␣
 ↪watching it several times.5. its sophistication - there is a lot of␣
 ↪information to remember and digest. This is the very thing the modern␣
 ↪moviegoers are after, I believe.6. its realism - okay, pun intended. The␣
 ↪movie explains (or at least tries to) the ins and outs of what dream is␣
 ↪about and how it functions, some of which are very familiar with and dear to␣
 ↪us.Cons: 1. its poor character development - although the acting was␣
 ↪convincing enough there was not enough of character development. I wonder␣
 ↪how many people really felt connected to the main character(s) after␣
 ↪watching the movie. Yes, the movie talks about emotional struggles but it␣
 ↪was more of an action film than anything else, if you ask me.2. too many␣
 ↪distractions - I found that the movie had more characters than necessary.␣
 ↪They may play certain roles in the plot but they seemed more of distractions␣
 ↪than anything else. I wish the movie was more focused.3. a bit preachy - I␣
 ↪noticed that the characters would explain things about dreamworld and then␣
 ↪the exact things happen later in the movie. I'm afraid, Inception overused␣
 ↪this trick.In conclusion, its theme is fascinating but its delivery is not␣
 ↪without room for improvement.I highly recommend you to go and read Somewhere␣
 ↪carnal over 40 winks, if you dig this kind of flicks.Cheers!"}
```

```
4864
{'rating': 10, 'Author': 'spaaw', 'review_date': '17 July 2010',
'review_content': "Many of us, not least those who populate the online world
with movie reviews, observations and criticisms, are aware of Chris Nolan's raw
talent. Memento broke onto the screen a decade ago and provided us with a
spectacle in story telling so rare that it dumbfounded us. Since then, Nolan has
gained huge popularity with his revival of the Batman franchise, and even more
so with the stunning Dark Knight - that still has yet to relinquish its grip on
modern day cinema - with the genius of The Prestige sandwiched in between.None
of this, however, could have quite prepared us for the spectacle that is
INCEPTION. A script that has been worked on for over a decade, Nolan's
endeavours are inescapable in its pure complexity and brilliance - quite simply,
one of the best film story lines audiences will have encountered in their
lifetime. All this is supplemented superbly by the film's ensemble cast -
```

```
DiCaprio's performance cannot easily be done justice without seeing it. With
this and Shutter Island, now must be the time for his first Oscar - he has
asserted his place as truly one of the best actors currently working. All
reviews will confirm that Joseph Gordon-Levitt, Tom Hardy and Ellen Page back
his performance up superbly with well-worked turns. Inception's action sequences
play out wonderfully, being - along with the comic relief and back-stories - an
integral part of the film's unravelling, whilst we remain rooted to the film's
emotional stakes, which give immense purpose to the film's complex and layered
narrative.Our protagonist battles with both outer and inner demons in his quest
for redemption. The unique concept of entering dreams offers the opportunity not
only for a unique species of heist but also a way to dig deeper and deeper into
one's subconscious, and the ability to plant an idea which could define that
person, or destroy them. In theaters around the world, Nolan uses his dynamic
skill to plant an idea in our minds which together allows us to experience the
very dream- share the film explores, one which will remain with us long after
the film's emotional climax. Inception is a cinema experience so audacious, so
incredible, and in the end so gratifying, that it truly is unmissable, and will
remain a benchmark for many filmmakers in years to come.10/10."}
```

1.13 Part 3 (10% of HW 1): Basic Probability (Refer to tutorial noteboook on Piazza for definition)

Now we will answer some basic probability questions. You may type the answers using markdown or attach photos of your answers using where image has your answers.

1. (5% credit) Let X and Y be random variables with the following joint distribution:

$\mathbb{P}(X = x, Y = y)$	X=1	X=2	X=3	X=4
Y=0	1/18	1/18	1/9	1/9
Y=1	1/12	1/12	1/6	1/15
Y=2	1/15	1/30	1/30	2/15

   Answer the following questions:

   1. What are the marginal distributions of X and Y?
   2. Are X and Y independent? Justify your answer.
   3. What is the conditional distribution of X, given that $Y = 2$? What is $\mathbb{E}[X|Y = 2]$?
   4. Calculate $\mathbb{E}[Y]$ and $\mathbb{E}[XY]$.

2. (5% credit) A coin is tossed three times with probability of heads p. Consider the following four events:

   A: Heads on the first toss

   B: Heads on the second toss

   C: All three outcomes the same

   D: Exactly two heads

Which of the following pairs of events are independent? (More than one pair may be independent.) Justify your answer.

1. A and B; 2. A and C; 3. A and D; 4. C and D

2 Submission

You're almost done!

After executing all commands and completing this notebook, save your *hw1.ipynb* as a pdf file and upload it to Gradescope under *Homework 1 (written)*. Make sure you check that your pdf file includes all parts of your solution **(including the outputs)**. We recommend using the browser (not jupyter) for saving the pdf. For Chrome on a Mac, this is under *File->Print…->Open PDF in Preview* and when the PDF opens in Preview you can use *Save…* to save it. This part will be graded based on completion (having executed the code and showing the output).

Next, you need to copy the functions from Questions 1.2 and 1.3 into the corresponding functions in *hw1part1.py*. Similarly, you need to copy the functions from Questions 2.1, 2.2, 2.3, 2.4, 2.5 and 2.6 into the corresponding functions in *hw1part2.py*. Place your files *hw1part1.py*, *hw1part2.py*, and *hw1.ipynb* in a zip file and upload the zip file to Gradescope under *Homework 1 - (code)*. In order to get full points for this part, you need to pass all test cases that we will run against your *hw1part1.py* and *hw1part2.py* (and not the notebook) on Gradescope. We have provided a sample of the test cases in *tests_sample_part1/tests.py* and *tests_sample_part2/tests.py*. Other tests are hidden on the Gradescope server. To check whether your code runs locally, run the four tests in *tests_sample_part1* from your command line:

```
(cs418-fa24) sathya@Sathyas-MacBook-Pro h1% python run_tests_sample.py part1
```

You should see the following output:

```
....
----------------------------------------------------------------------
Ran 4 tests in 0.001s

OK
```

Feel free to add more tests that check all parts of your code.

Similarly, you can run sample tests for part2 as follows:

```
(cs418-fa24) sathya@Sathyas-MacBook-Pro h1% python run_tests_sample.py part2
```

You can submit to Gradescope as many times as you would like. We will only consider your last submission. If your last submission is after the deadline, the late homework policy applies.

After submitting the zip file, the autograder will run. If you see a screen after the autograder finishes the execution with all correct, then it means that all the tests ran successfully on the server, and you're done! If your tests fail, you can debug your program locally by comparing the input, output and expected output (as shown for first two test cases)..

Make sure `hw1part1.py`, `hw1part2.py` and `hw1.ipynb` are included on the root of the zip file. **This means you need to zip those files and not the folder containing the files.**

1.1 What are the marginal distributions of X and Y?

$P(X=1) = P(X=1, Y=0) + P(X=1, Y=1) + P(X=1, Y=2)$

$= \frac{1}{18} + \frac{1}{12} + \frac{1}{15}$

$= \frac{37}{180}$

$P(X=2) = P(X=2, Y=0) + P(X=2, Y=1) + P(X=2, Y=2)$

$= \frac{1}{18} + \frac{1}{12} + \frac{1}{30}$

$= \frac{31}{180}$

$P(X=3) = P(X=3, Y=0) + P(X=3, Y=1) + P(X=3, Y=2)$

$= \frac{1}{9} + \frac{1}{6} + \frac{1}{30}$

$= \frac{14}{45}$

$P(X=4) = P(X=4, Y=0) + P(X=4, Y=1) + P(X=4, Y=2)$

$= \frac{1}{9} + \frac{1}{15} + \frac{2}{15} = \frac{14}{45}$

$P(Y=0) = P(X=0, X=1) + P(Y=0, X=2) + P(Y=0, X=3) + P(Y=0, X=4)$

$= \frac{1}{18} + \frac{1}{18} + \frac{1}{9} + \frac{1}{9} = \frac{1}{3}$

$P(Y=1) = P(Y=1, X=1) + P(Y=1, X=2) + P(Y=1, X=3) + P(Y=1, X=4)$

$= \frac{1}{12} + \frac{1}{12} + \frac{1}{6} + \frac{1}{15} = \frac{2}{5}$

$P(Y=2) = P(Y=2, X=1) + P(Y=2, X=2) + P(Y=2, X=3) + P(Y=2, X=4)$

$= \frac{1}{15} + \frac{1}{30} + \frac{1}{30} + \frac{2}{15} = \frac{4}{15}$

$P(X=1) = \frac{37}{180}$

$P(X=2) = \frac{31}{180}$

$P(X=3) = \frac{15}{45}$

$P(X=4) = \frac{14}{45}$

$P(Y=0) = \frac{1}{3}$

$P(Y=1) = \frac{2}{5}$

$P(Y=2) = \frac{4}{15}$

1.2 Are X and Y independent? Justify your answer.

$P(X=1, Y=0) = \frac{1}{18} \neq P(X=1) \cdot P(Y=0) = \frac{37}{540}$

X and Y are not independent.

1.3. What is the conditional distribution of X, given that Y=2?

what is $E[X|Y=2]$?

$$P(X=1|Y=2) = \frac{P(X=1, Y=2)}{P(Y=2)} = \frac{1}{15} \times \frac{15}{4} = \frac{1}{4}$$

$$P(X=2|Y=2) = \frac{P(X=2, Y=2)}{P(Y=2)} = \frac{1}{30} \times \frac{15}{4} = \frac{1}{8}$$

$$P(X=3|Y=2) = \frac{P(X=3, Y=2)}{P(Y=2)} = \frac{1}{30} \times \frac{15}{4} = \frac{1}{8}$$

$$P(X=4|Y=2) = \frac{P(X=4, Y=2)}{P(Y=2)} = \frac{2}{15} \times \frac{15}{4} = \frac{1}{2}$$

$$E(X|Y=2) = 1 \cdot P(X=1|Y=2) + 2 \cdot P(X=2|Y=2) + 3 \cdot P(X=3|Y=2) + 4 \cdot P(X=4, Y=2)$$

$$= 1 \cdot \frac{1}{4} + 2 \cdot \frac{1}{8} + 3 \cdot \frac{1}{8} + 4 \cdot \frac{1}{2}$$

$$= \frac{1}{4} + \frac{1}{4} + \frac{3}{8} + 2$$

$$= \frac{23}{8}$$

1.4. Calculate $E(Y)$ and $E(XY)$.

$$E(Y) = 0 \cdot P(Y=0) + 1 \cdot P(Y=1) + 2 \cdot P(Y=2) = 0 \cdot \frac{1}{3} + 1 \cdot \frac{2}{5} + 2 \cdot \frac{4}{15} = \frac{14}{15}$$

$$E(XY) = 0 + 1 \times \frac{1}{12} + 2 \times \frac{1}{12} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{15} + 2 \cdot \frac{1}{15} + 4 \cdot \frac{1}{30} + 6 \cdot \frac{1}{30} + 8 \cdot \frac{2}{15}$$

$$= \frac{1}{12} + \frac{1}{6} + \frac{1}{2} + \frac{4}{15} + \frac{2}{15} + \frac{2}{15} + \frac{3}{15} + \frac{16}{15}$$

$$= \frac{51}{20}$$

2. 1.4

assum the probability of head on the first toss is P

$P(A)=P \quad P(B)=P \quad P(C)=P^3+(1-P)^3 \quad P(D)=3P^2(1-P)$

1. A and B

$P(A)\cdot P(B)=P^2=P(AB)=P^2$ A and B are independant

2. A and C

$P(A)\cdot P(C)=P\cdot[P^3+(1-P)^3]$

$P(A\cdot C)=P^3$ A and C are not independent

$P(A)\cdot P(C)\neq P(A\cdot C)$

3. A and D

$P(A)\cdot P(D)=3P^3(1-P)$

$P(A\cdot D)=2P^2(1-P)$ A and D are not independant

$P(A)\cdot P(D)\neq P(A\cdot D)$

4. C and D

~~$P(C)\cdot P(D)=$~~

$P(C\cap D)=0$ C and D are independent

	30111 Dire	30112 Direction
0	-0.51773	0.068493
1	-0.11409	0.463087
2	-0.11724	0.387755
3	-0.44554	0.223301
4	-0.20863	0.569444
5	-1.05607	0.113043
6	-0.88182	0.45283
Variance	0.139167	0.036694